Model training progress

English | 中文

Reviewed November 12, 2026. This report describes saved training and model-validation records. It complements the November 11 project progress report.

Training and data

Task	Model	Training / validation images	Epochs completed	Training steps
Glaucoma A	EfficientNet-B0	10,354 / 1,190	20 / 20	12,960
Glaucoma B, candidate	ResNet50	10,354 / 1,190	20 / 20	12,960
Original quality model	EfficientNet-B0	12,147 / 1,642	20 / 20	15,200
EyeQ quality model	EfficientNet-B0	4,722 / 616	20 / 20	5,920

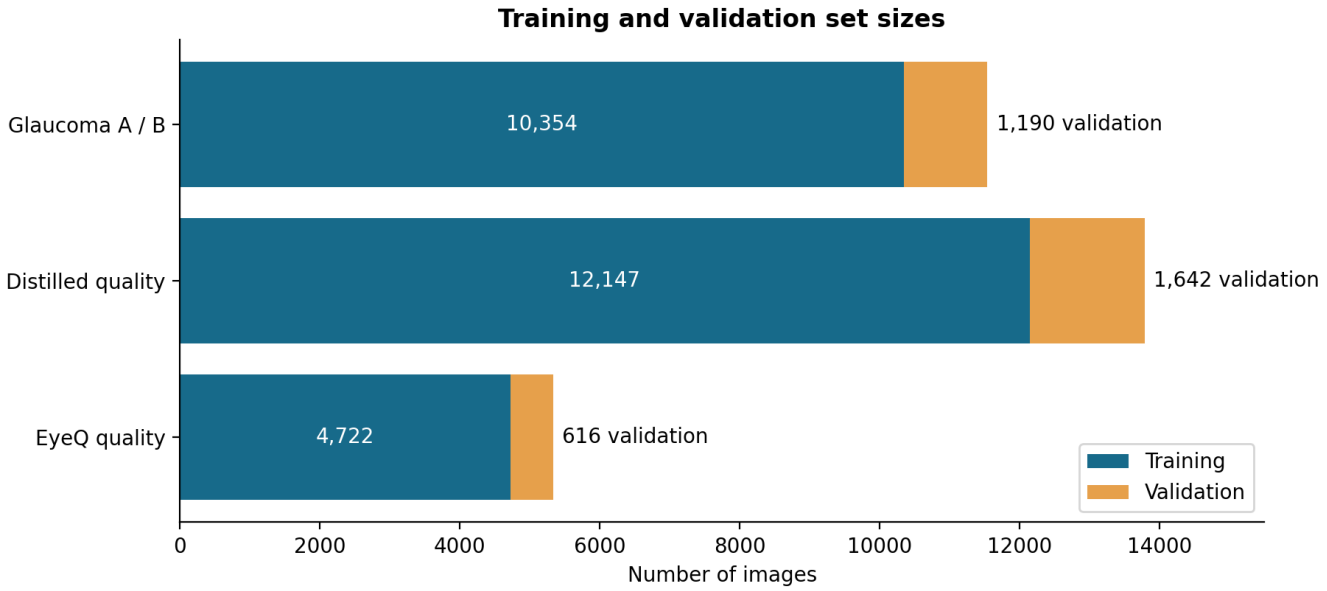


Figure 1: Training and validation set sizes

Both training pipelines report completion. An epoch is one pass through the training data; a step uses one batch of images to update the model. The two glaucoma models use the same data split. Each run records a loss at every training step and validation results after each epoch. Loss measures the difference between model output and its training target; lower is better for that target.

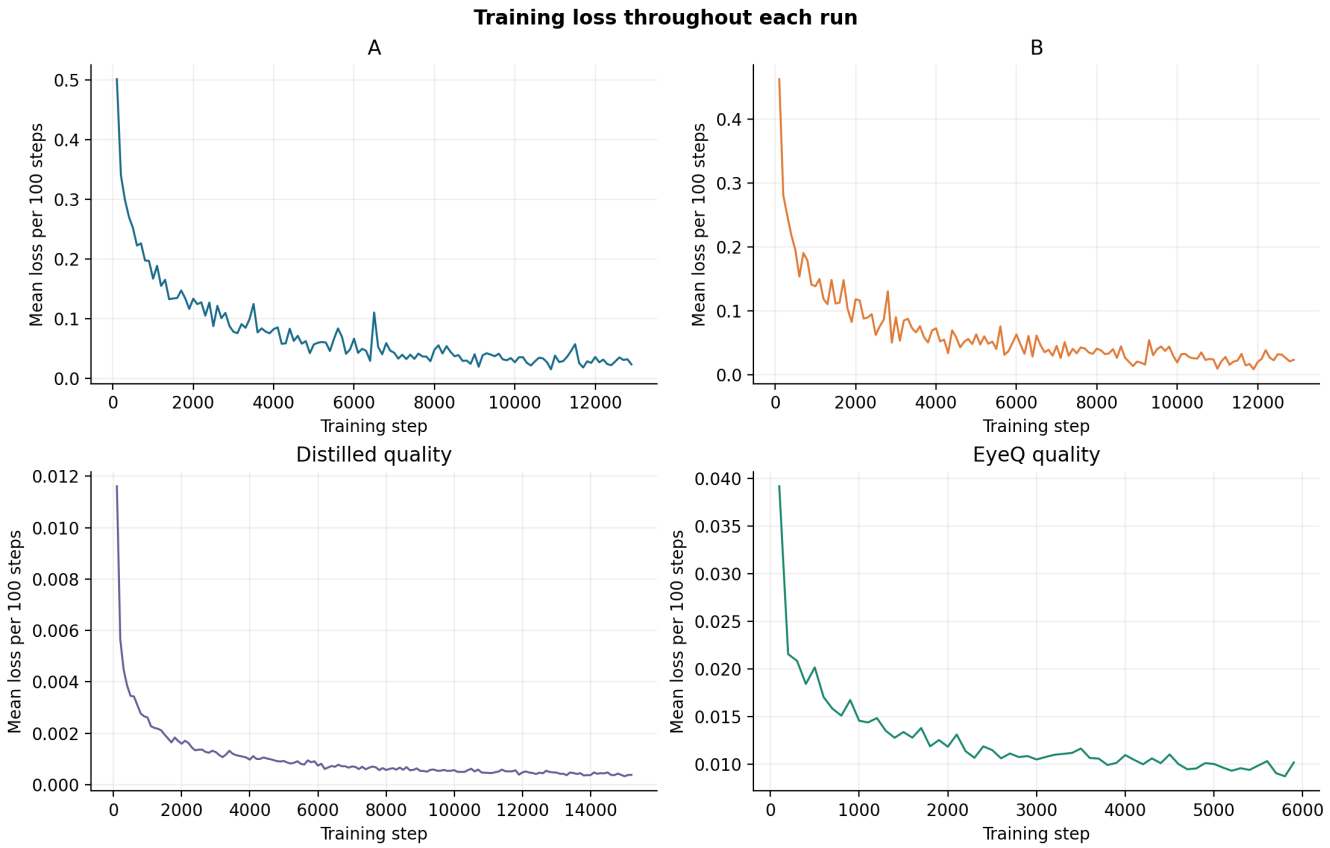


Figure 2: Training loss by step

Each point is the mean loss across 100 consecutive steps. All four step logs are continuous and their recorded losses are finite.

Glaucoma models

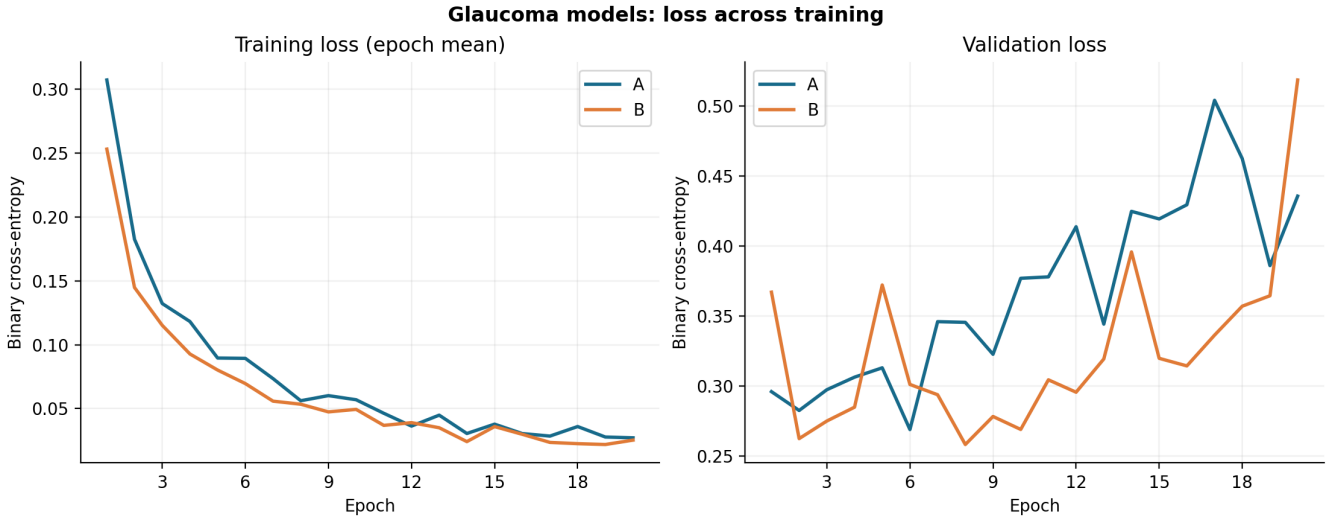


Figure 3: Glaucoma losses

Mean training loss fell from 0.307 to 0.027 for A and from 0.253 to 0.025 for B between the first and final epochs. Overall validation loss rose from 0.296 to 0.435 for A and from 0.367 to 0.518 for B. This divergence warrants a closer look at overfitting and probability calibration. Training curves show the arithmetic mean of batch losses within each epoch.

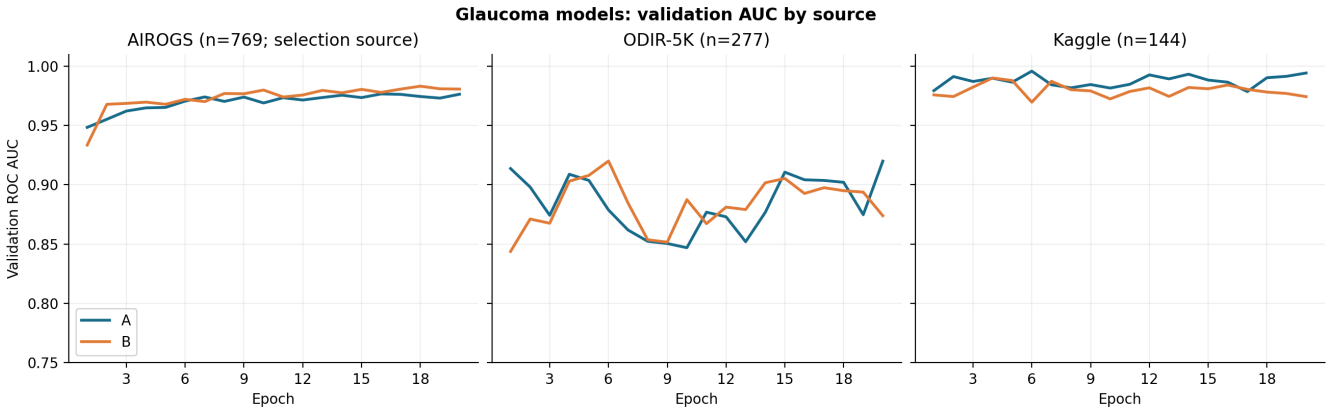


Figure 4: Validation AUC by source

AUC measures how well the model ranks positive cases above negative cases, without selecting a classification threshold. The AIROGS validation subset (769 images) was used to select checkpoints: A reached its best AUC of **0.9766 at epoch 16**, and B reached **0.9831 at epoch 18**. At epoch 20 the values were 0.9764 and 0.9807, respectively. ODIR-5K and Kaggle validation subsets contain 277 and 144 images.

The disease-model logs do not contain accuracy for each epoch. The prediction file is overwritten at each validation, so it represents the last validation, not every epoch or the selected checkpoint.

Image quality models

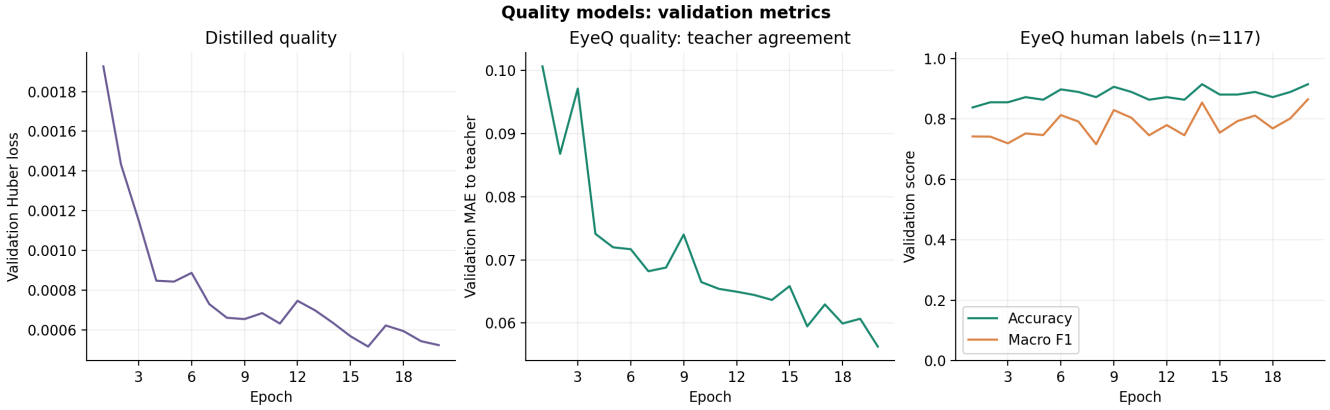


Figure 5: Quality validation measures

The original quality model's validation Huber loss fell from 0.001927 to 0.000524; its minimum was 0.000517 at epoch 16. This model learns continuous scores from a teacher model. Its validation target is the teacher score.

The EyeQ model was validated on 616 images, including 117 with human grades in three categories. Accuracy calculated from the saved confusion matrices rose from **83.8% (98/117)** at epoch 1 to **91.5% (107/117)** at epoch 20. The final macro-F1 was **0.865**. Macro-F1 gives equal weight to the performance of each grade, even when the grade counts differ.

The selected EyeQ checkpoint is epoch 17, chosen using a combined teacher and human-grade validation loss. Its accuracy is **88.9% (104/117)** and macro-F1 is **0.811**. It is a different checkpoint from the final epoch.

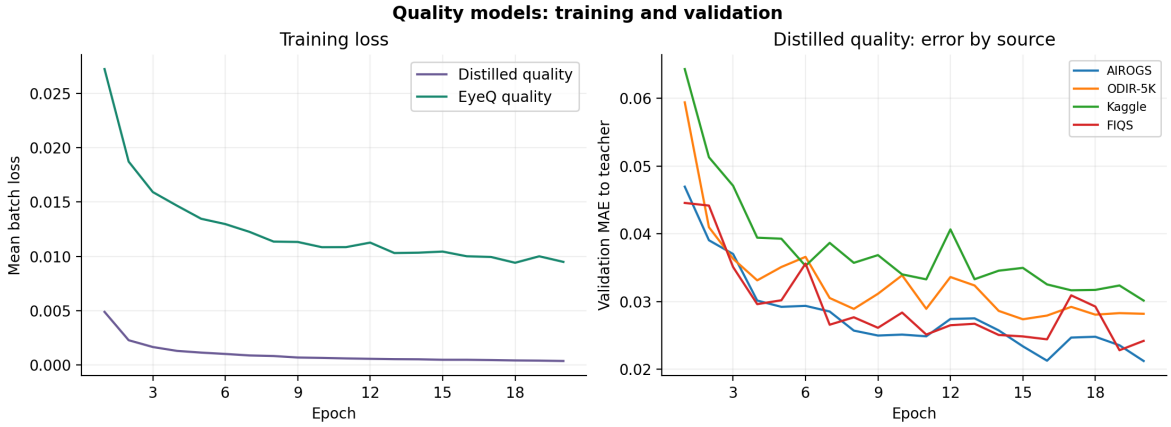


Figure 6: Quality training and error by source

The left panel shows each quality model's training loss. The right panel shows the original model's mean absolute difference from the teacher score by image source. Lower values indicate closer agreement with the teacher.

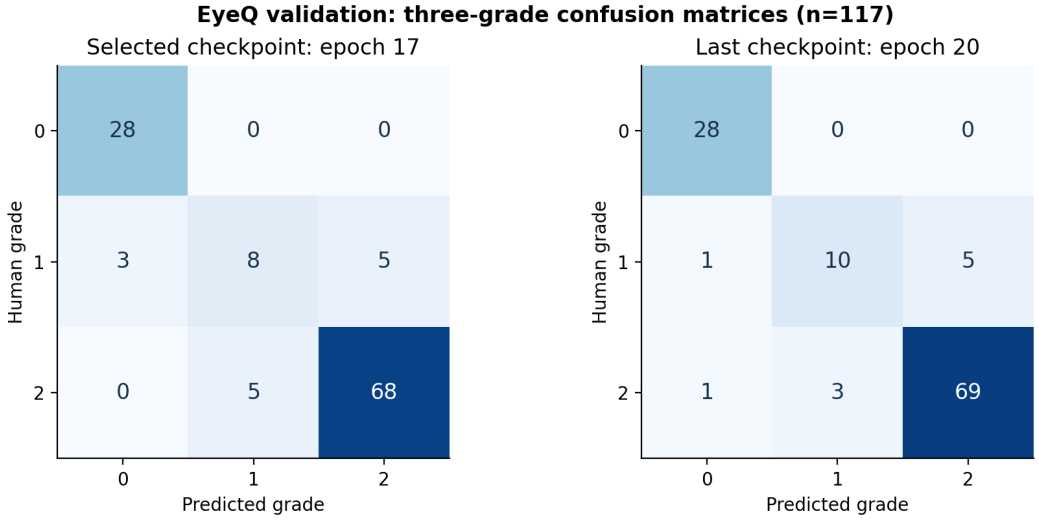


Figure 7: EyeQ confusion matrices

Rows show the human grade and columns show the predicted grade. The diagonal contains correct predictions. The left matrix is the selected checkpoint; the right matrix is the last epoch.

Data and figure generation

The source records are the step and epoch logs, configurations, run summaries, and pipeline status files. The glaucoma and original quality model records are under `runs/formal_v3/`; EyeQ records are under `runs/eyeq_quality_v1/quality_student/`. In the original formal_v3 logs, epoch numbers start at zero; the figures use 1–20. EyeQ validation logs already use 1–20.

The accompanying `chart_data.json` contains step losses and aggregate validation metrics, without images or sample identities. Run `plot_training.py` with Python and matplotlib to recreate the figures from this snapshot.